

Chapter 3

Transport Layer

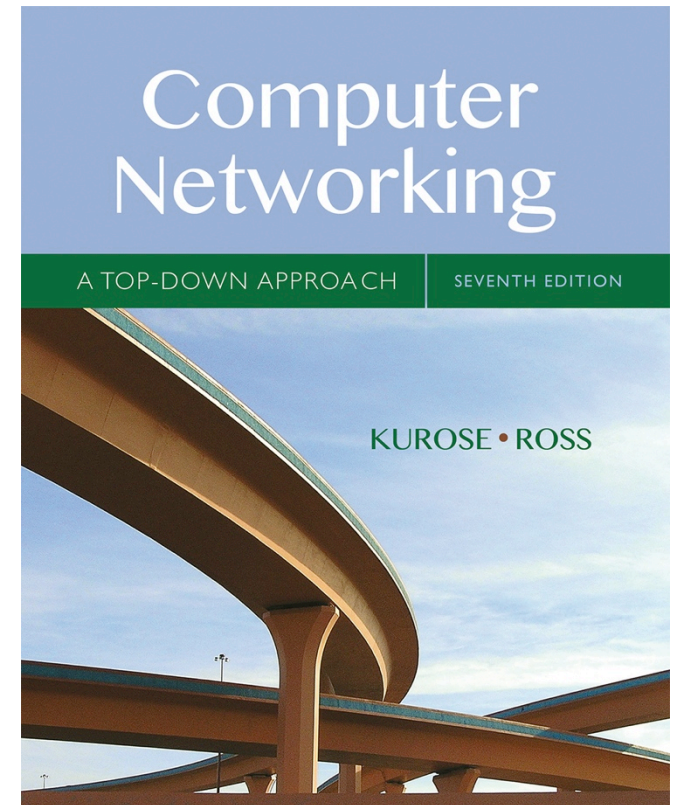
A note on the use of these Powerpoint slides:

We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

© All material copyright 1996-2016
J.F Kurose and K.W. Ross, All Rights Reserved



Computer Networking: A Top Down Approach

7th edition

Jim Kurose, Keith Ross

Pearson/Addison Wesley

April 2016

Chapter 3: Transport Layer

our goals:

- understand principles behind transport layer services:
 - multiplexing, demultiplexing
 - reliable data transfer
 - flow control
 - congestion control
- learn about Internet transport layer protocols:
 - UDP: connectionless transport
 - TCP: connection-oriented reliable transport
 - TCP congestion control

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

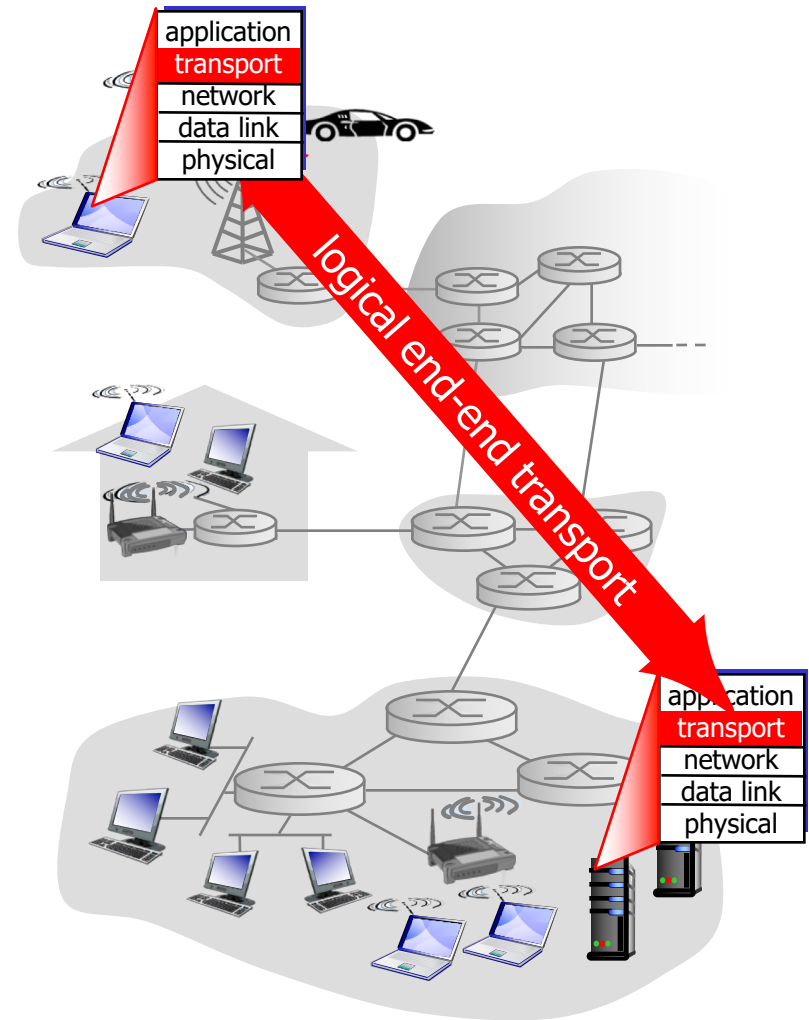
- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

Transport services and protocols

- provide *logical communication* between app processes running on different hosts
- transport protocols run in end systems
 - send side: breaks app messages into *segments*, passes to network layer
 - rcv side: reassembles segments into messages, passes to app layer
- more than one transport protocol available to apps
 - Internet: TCP and UDP



Transport vs. network layer

- *network layer*: logical communication between end devices (hosts)
- *transport layer*: logical communication between processes
 - relies on, enhances, network layer services

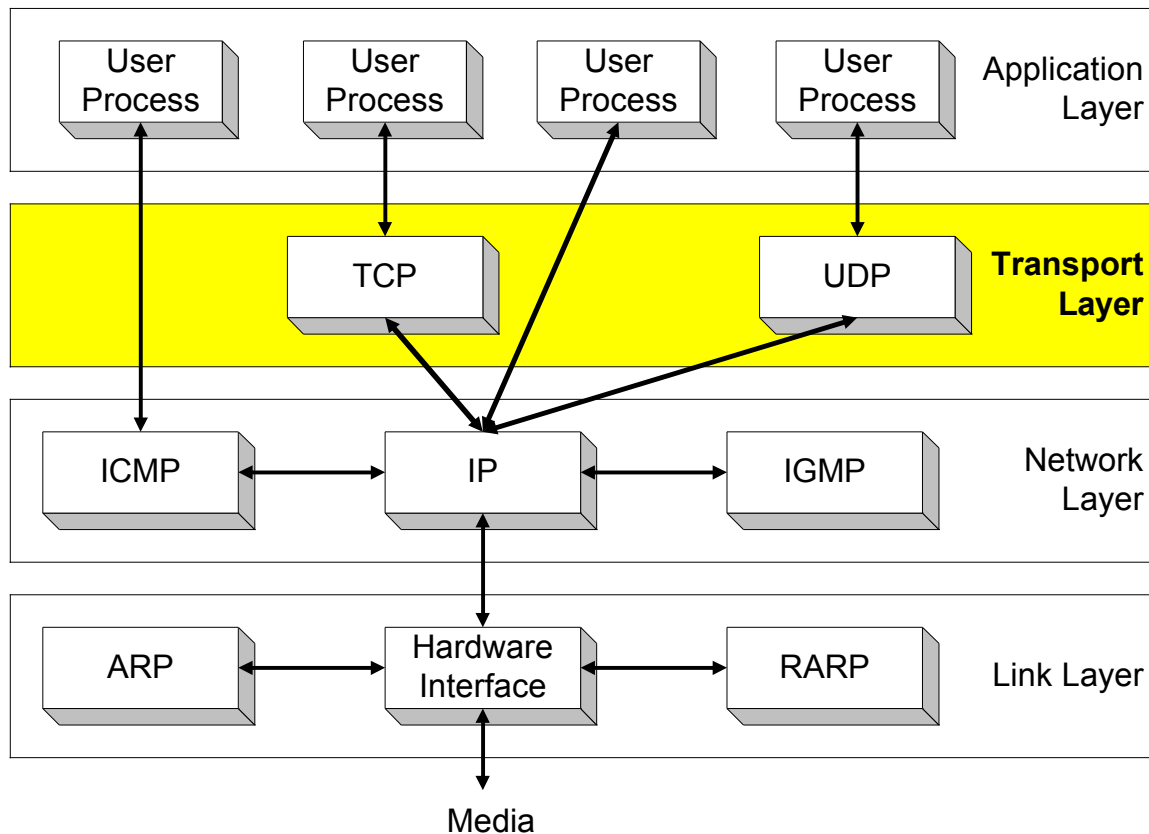
human analogy:

Postal service – sending and receiving letters to homes:

- **hosts** => homes
- **processes** => people living in the homes
- **application messages** => letters in envelopes
- **transport protocol** => sort the letters by name to the people in the homes
- **network-layer protocol** => postal service delivery

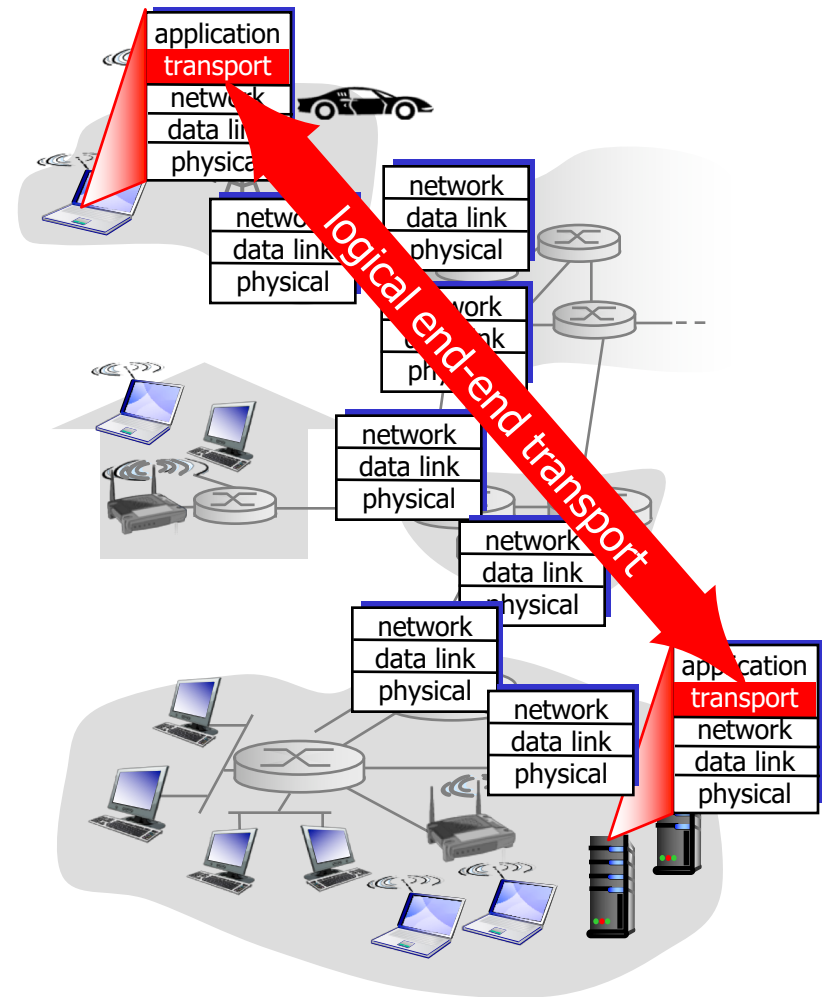
Orientation

- Where is the transport layer?



Internet transport-layer protocols

- reliable, in-order delivery (TCP)
 - congestion control
 - flow control
 - connection setup
- unreliable, unordered delivery: UDP
 - no-frills extension of “best-effort” IP
- services **not** available:
 - delay guarantees
 - bandwidth guarantees



Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

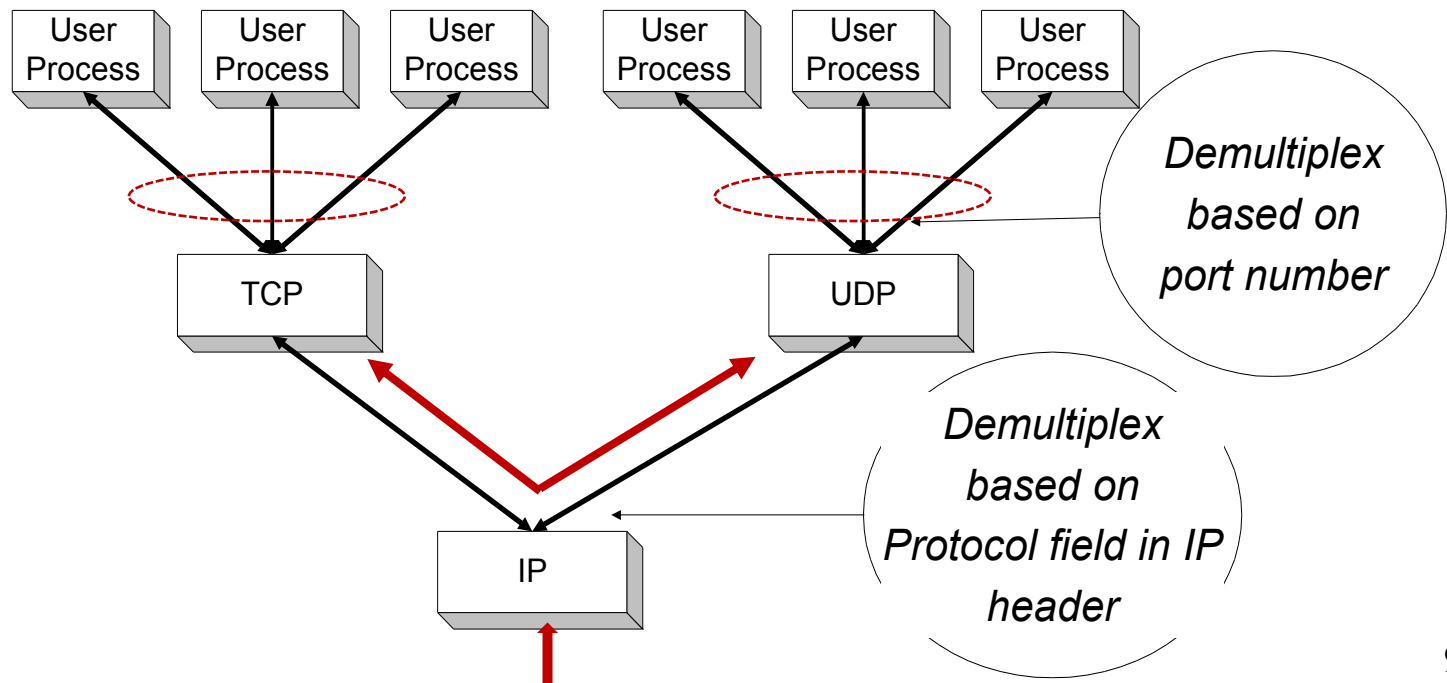
- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

Port Numbers

- UDP and TCP use port numbers to identify connections between **end user processes**
- A globally unique address at the transport layer (for both UDP and TCP) is the tuple **<IP address, port number>**
- 16 bits are reserved for port numbers in the header --> $2^{16} = 65,535$ ports per host.



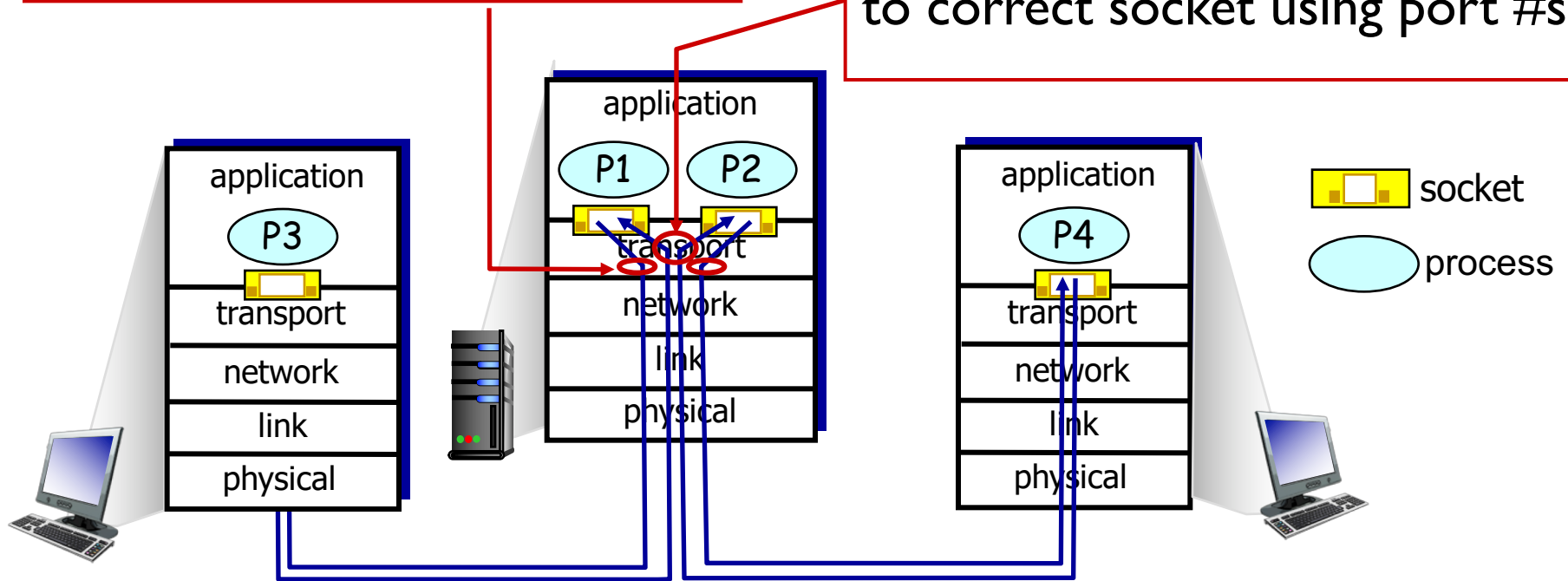
Multiplexing/demultiplexing

multiplexing at sender:

handle data from multiple Sockets (P1,P2), add transport header (includes port #s for demultiplexing)

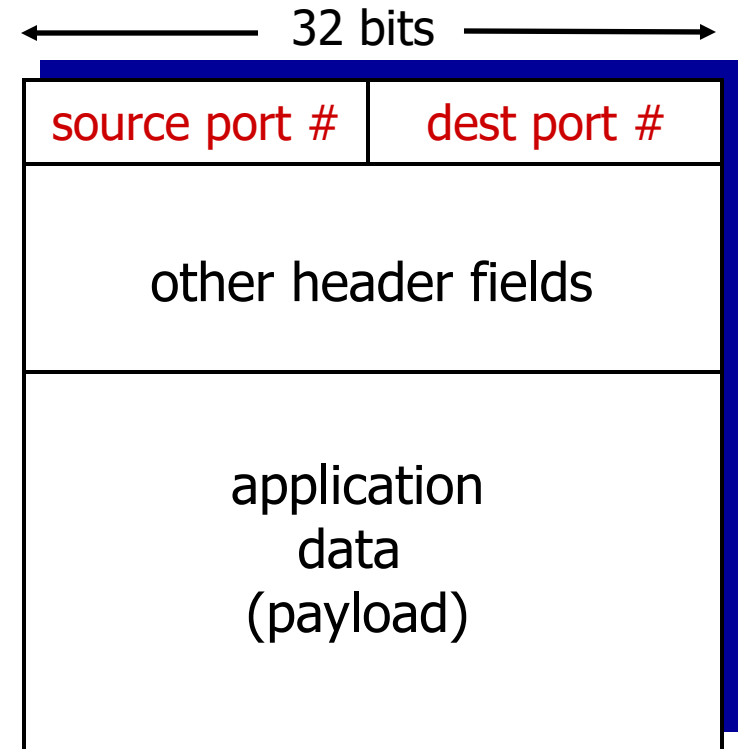
demultiplexing at receiver:

use header info to deliver received segments from P3, P4 to correct socket using port #s



How demultiplexing works

- host receives IP datagrams
 - each datagram has source IP address, destination IP address and protocol type
 - each datagram carries one transport-layer segment
 - each segment has source and destination port number
- host uses *IP addresses, protocol field & port numbers* to receive data and direct segment to appropriate socket in transport layer (TCP or UDP)

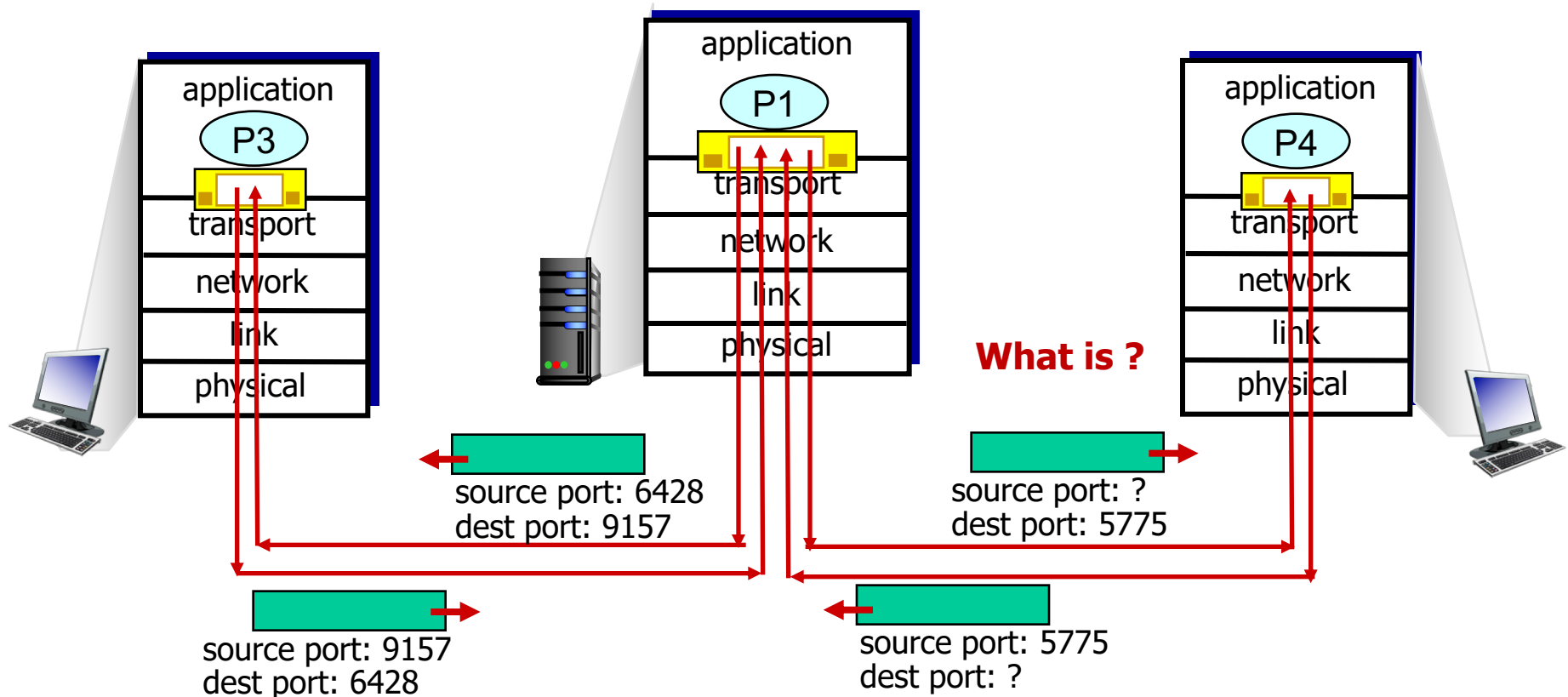


TCP/UDP segment format

Connectionless (UDP) demultiplexing

- create a socket w host-local (origin) port #:
`DatagramSocket mySocket1
= new DatagramSocket(12534) ;`
 - when creating datagram to send into UDP socket, must specify
 - destination IP address
 - destination port #
-
- when host receives UDP segment:
 - checks destination port # in segment
 - directs UDP segment to socket with that port #
- ➔
- IP datagrams with *same dest. port #*, but different source IP addresses *and/or* source port numbers will be directed to *same socket* at destination

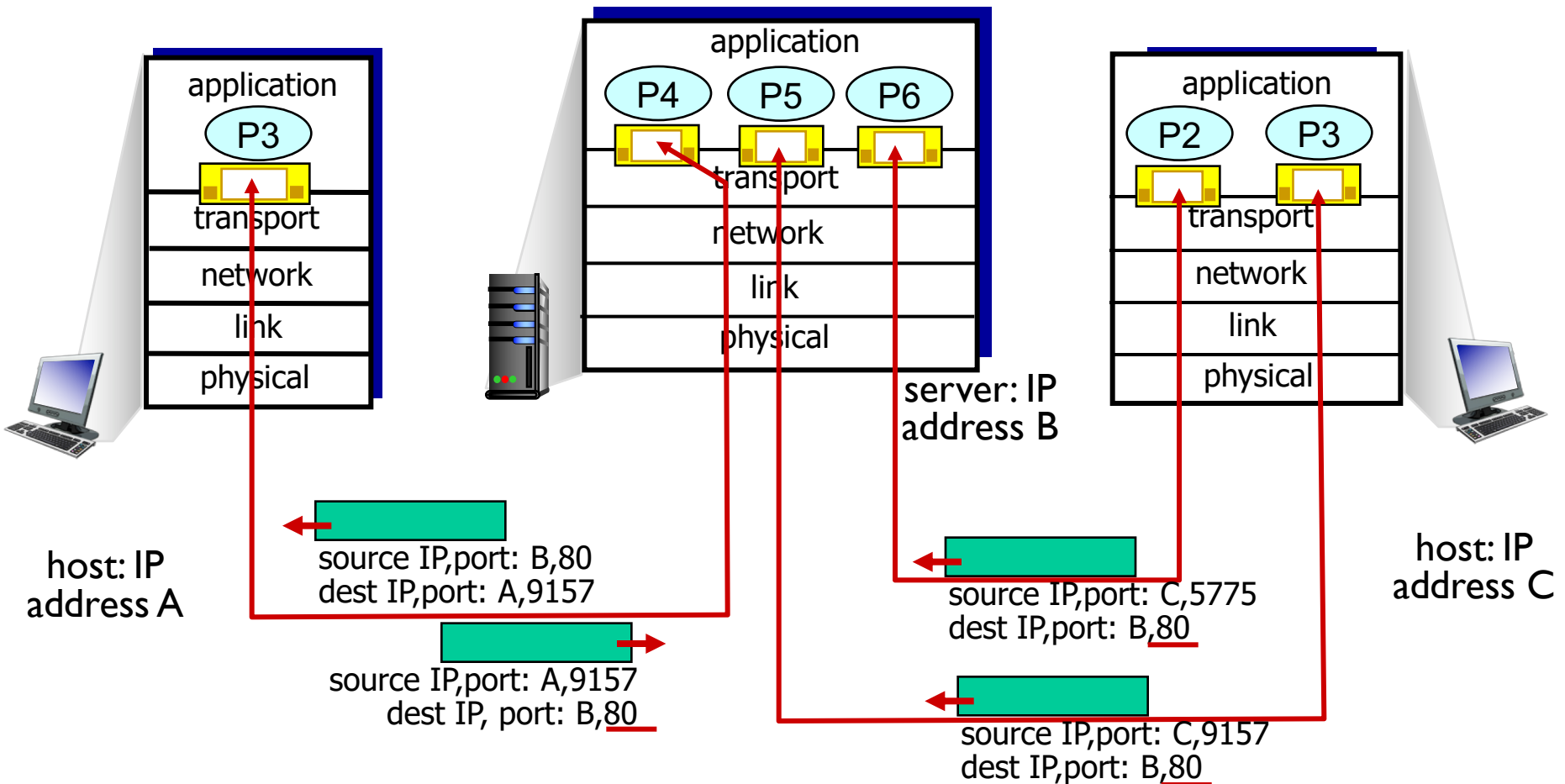
Connectionless demux: example



Connection-oriented TCP demux

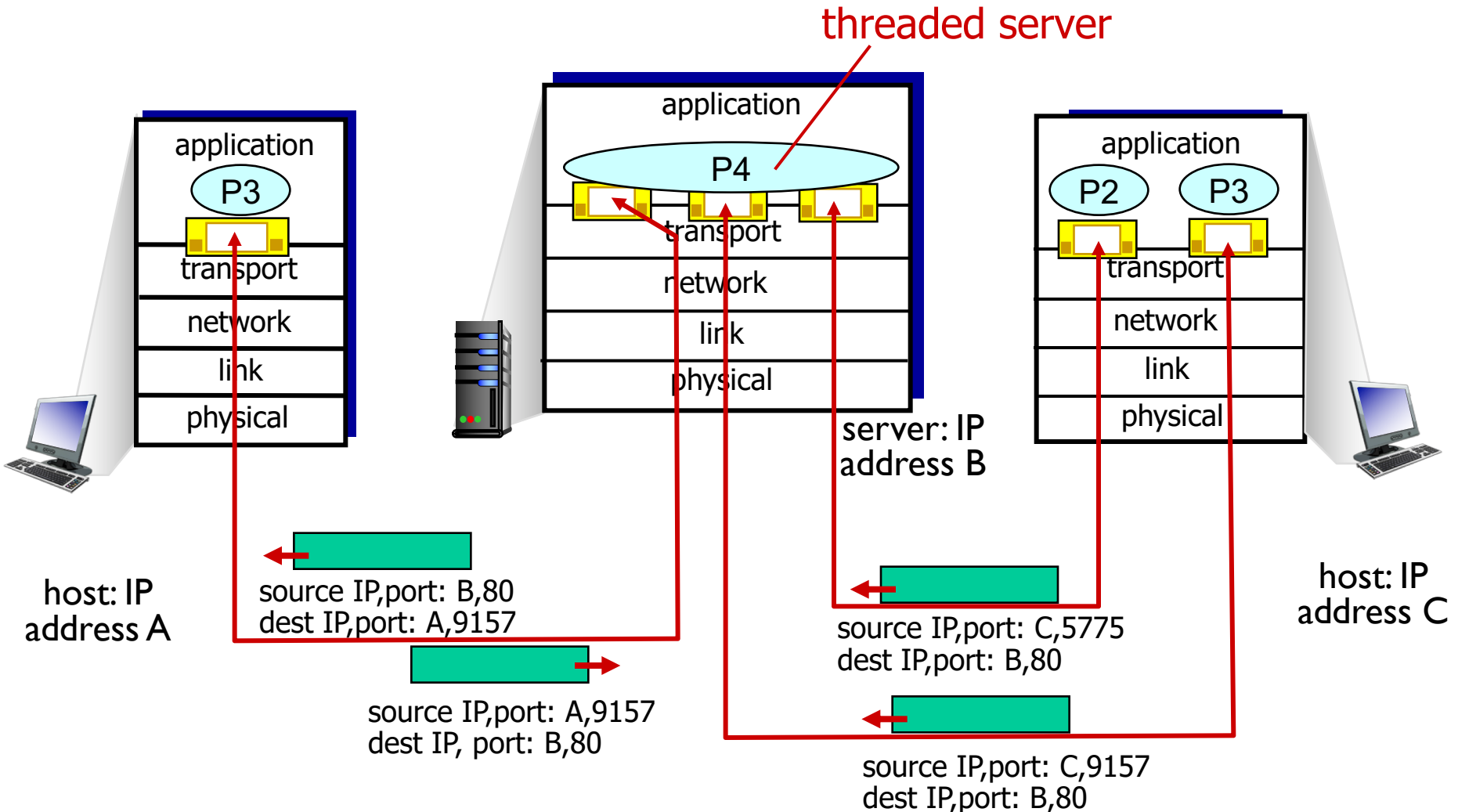
- TCP socket identified by 4-tuple:
 - source IP address
 - source port number
 - dest IP address
 - dest port number
- demux: receiver uses all four values to direct segment to appropriate socket
- server host may support many simultaneous TCP sockets:
 - each socket identified by its own 4-tuple
- e.g., web servers have different sockets for each connecting client
 - e.g., (non-persistent) HTTP will have different socket for each request

Connection-oriented demux: example



three segments, all destined to IP address: B,
dest port: 80 are demultiplexed to *different* sockets

Connection-oriented demux: example



Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

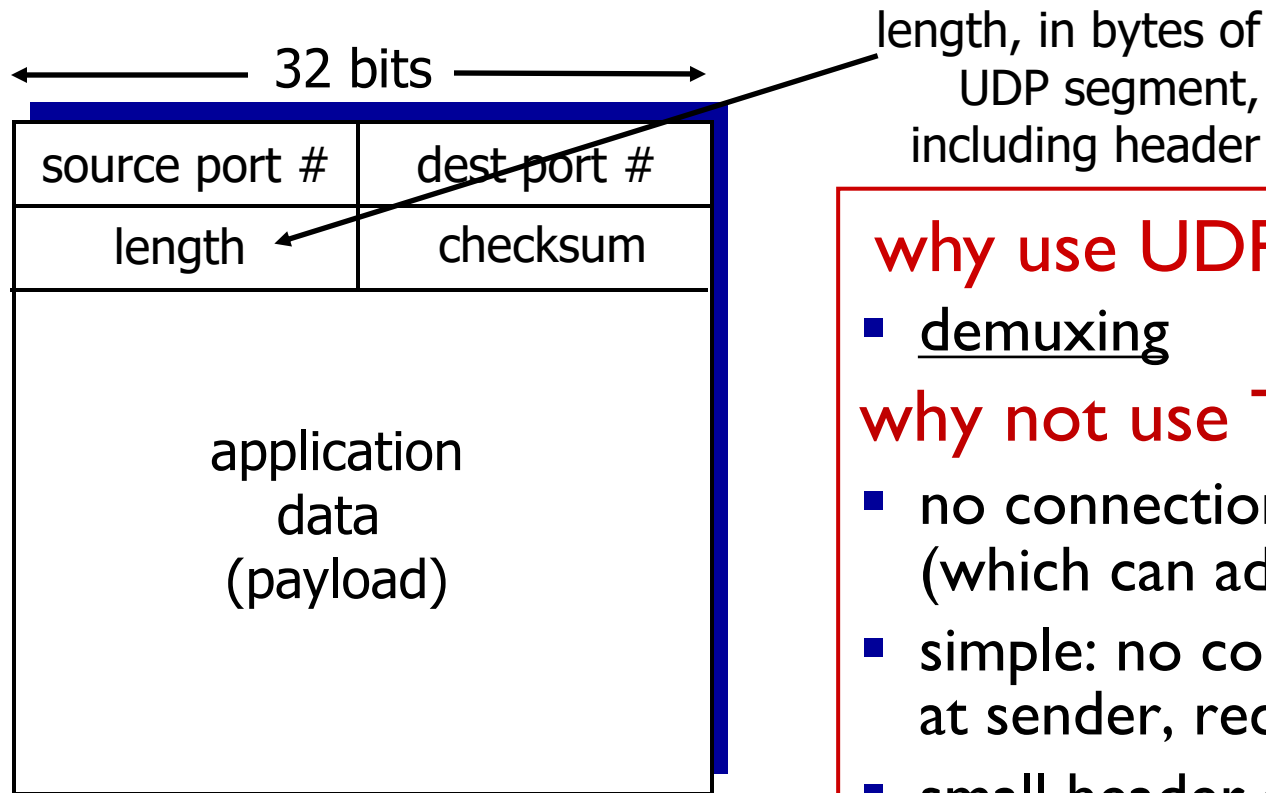
3.6 principles of congestion control

3.7 TCP congestion control

UDP: User Datagram Protocol [RFC 768]

- “no frills,” “bare bones”
Internet transport
protocol
- “best effort” service, UDP
segments may be:
 - lost
 - delivered out-of-order
to app
- *connectionless*:
 - no handshaking
between UDP sender,
receiver
 - each UDP segment
handled independently
of others
- UDP use:
 - streaming multimedia
apps (loss tolerant, rate
sensitive)
 - DNS
 - SNMP
- need **reliable** transfer
over UDP?
 - add reliability at
application layer
 - application-specific error
recovery!

UDP: segment header



UDP segment format

why use UDP?

- demuxing

why not use TCP?

- no connection establishment (which can add delay)
- simple: no connection state at sender, receiver
- small header size
- no congestion control: UDP can blast away as fast as desired

UDP checksum

Goal: detect “errors” (e.g., flipped bits) in transmitted segment

sender:

- treat segment contents, including header fields, as sequence of 16-bit integers
- checksum: one's complement sum of segment contents
- sender puts checksum value into UDP checksum field

receiver:

- compute checksum of received segment (one's complement of received segment content)
- computed checksum **equals** checksum field value??
 - NO - error detected, drop segment
 - YES - no error detected
But maybe errors nonetheless? Yes!

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

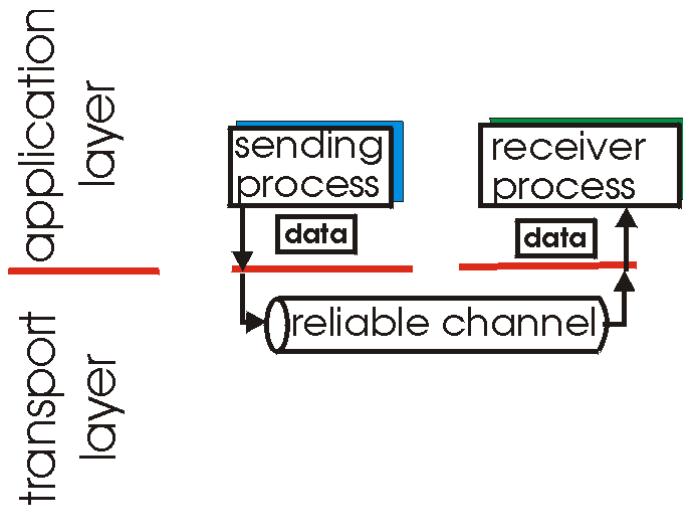
3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

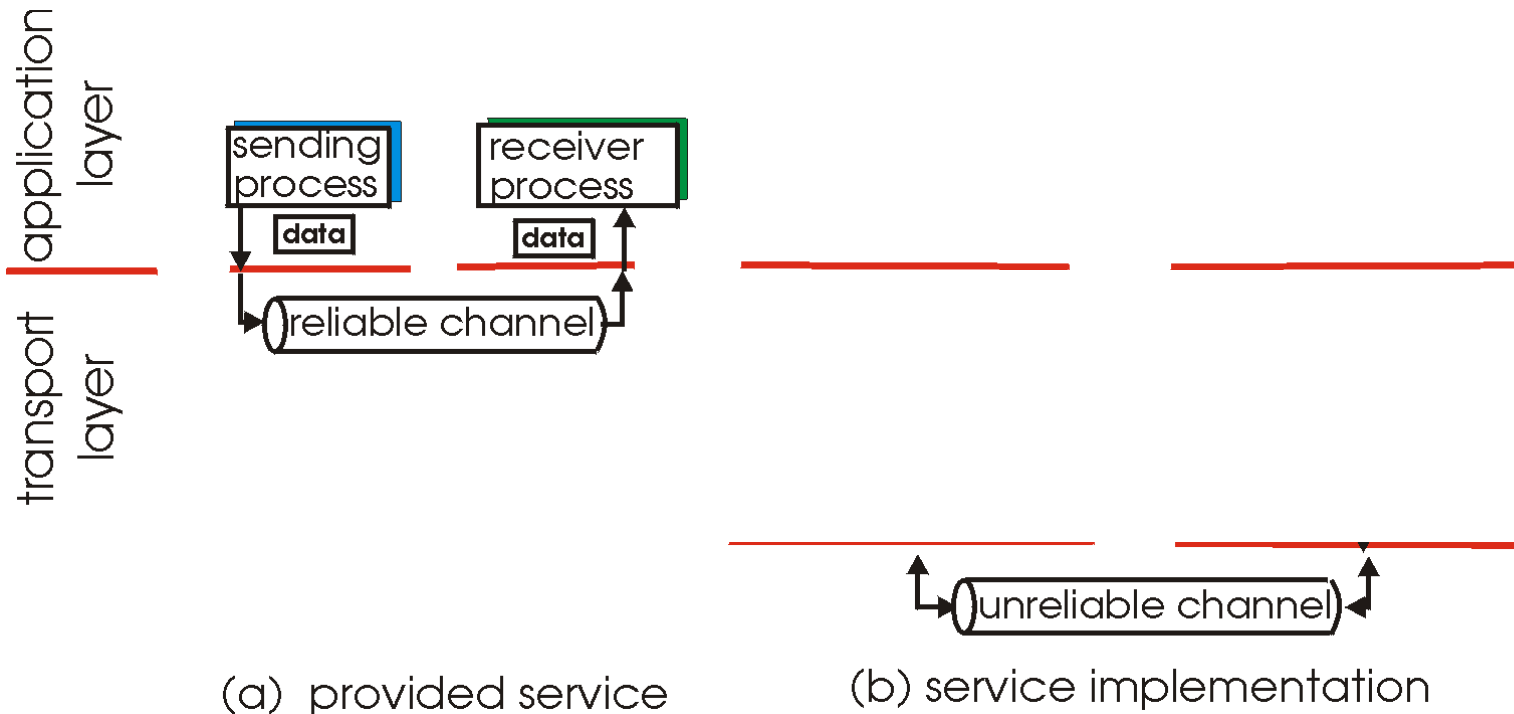
Principles of reliable data transfer



(a) provided service

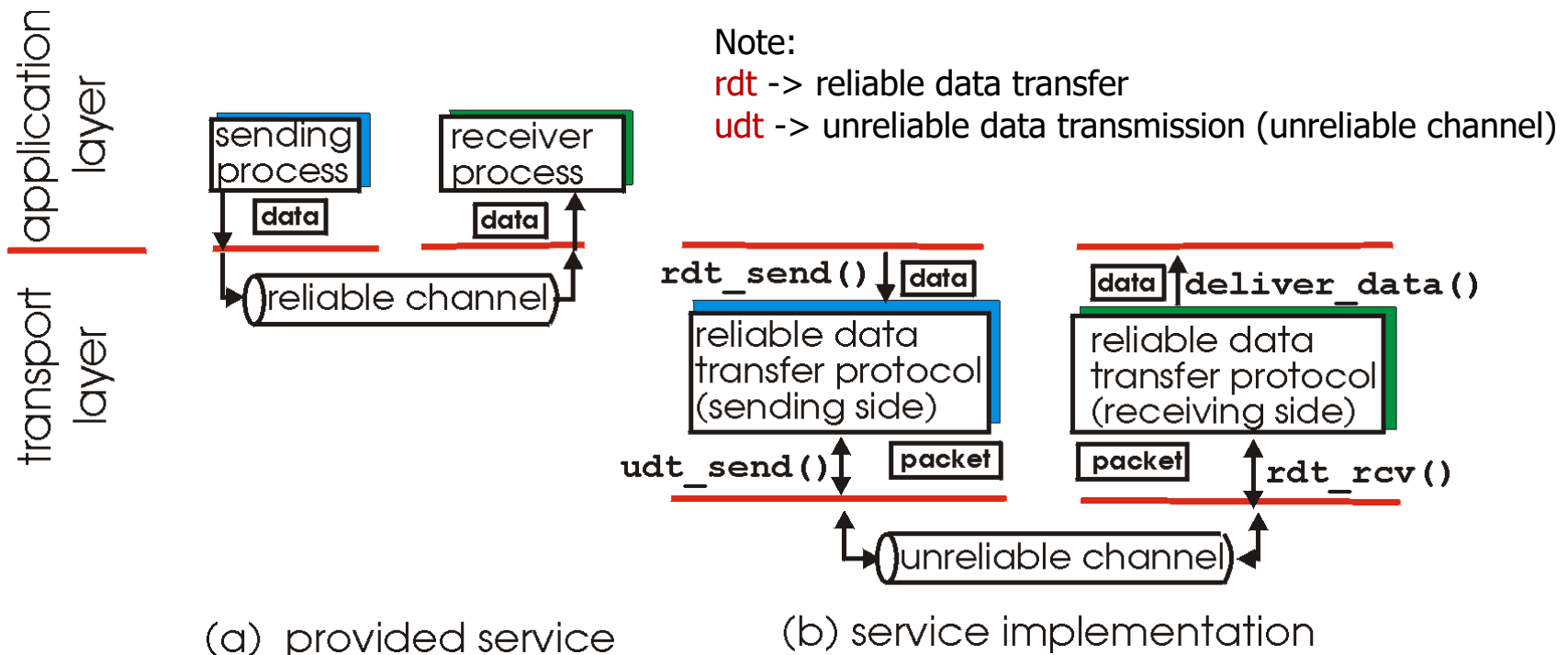
- **characteristics** of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

Principles of reliable data transfer



- **characteristics** of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

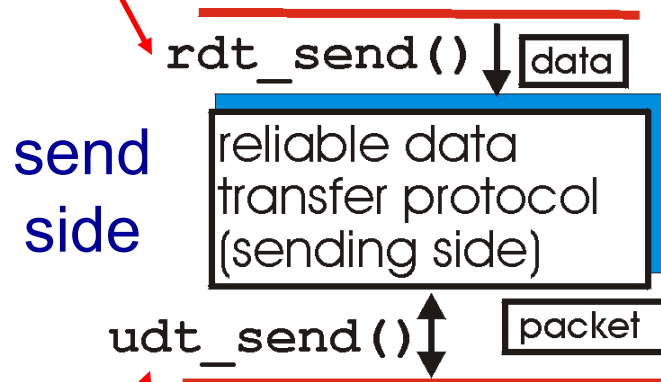
Principles of reliable data transfer



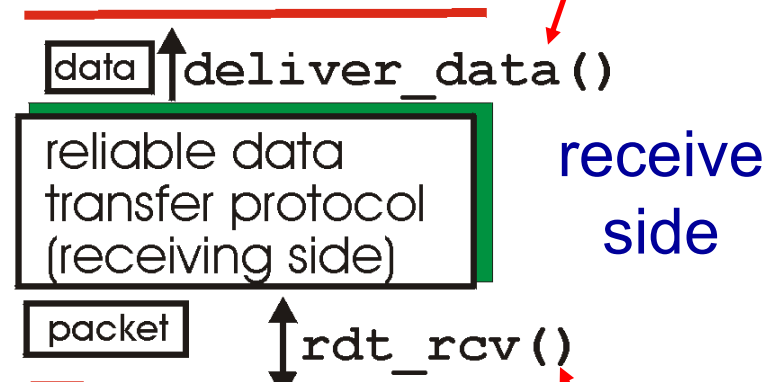
- **characteristics** of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

Reliable data transfer (rdt): getting started

rdt_send() : called from above, (e.g., by app.). Passes data for delivery to receiver upper layer



deliver_data() : called by **rdt** to deliver received data to upper layer



udt_send() : called by rdt, to transfer packet over **unreliable** channel to receiver

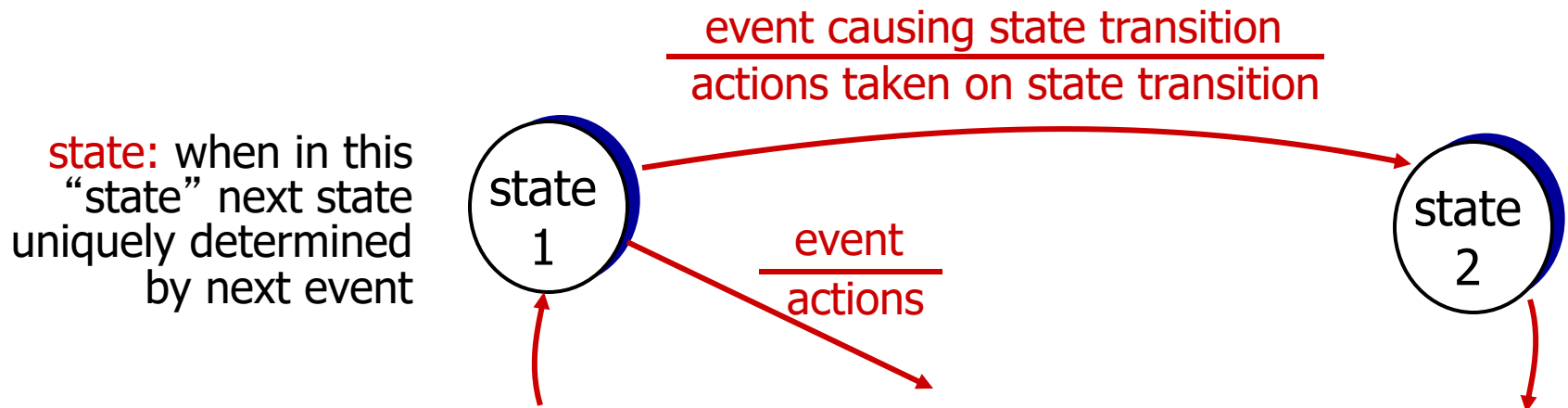
rdt_rcv() : called when packet arrives on rcv-side of channel



Reliable data transfer: getting started

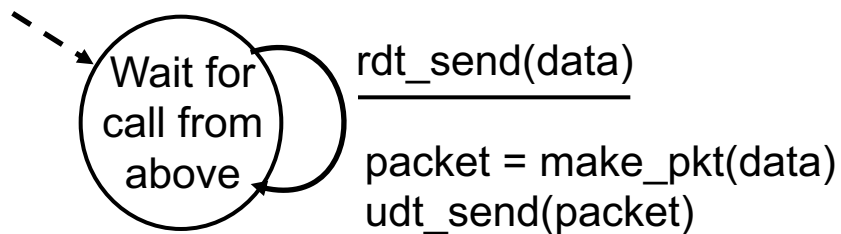
We will:

- incrementally develop sender, receiver sides of reliable data transfer protocol (rdt)
- consider only unidirectional data transfer
 - but control info will flow in both directions!
- use finite state machines (FSM) to specify sender, receiver

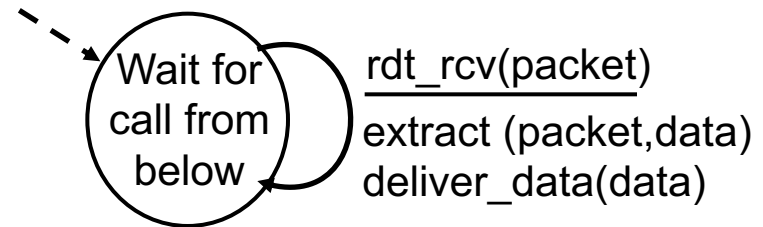


rdt1.0: reliable transfer over a reliable channel

- underlying channel perfectly reliable
 - no bit errors
 - no loss of packets
- separate FSMs for sender, receiver:
 - sender sends data into underlying channel
 - receiver reads data from underlying channel



sender



receiver

rdt2.0: channel with bit errors

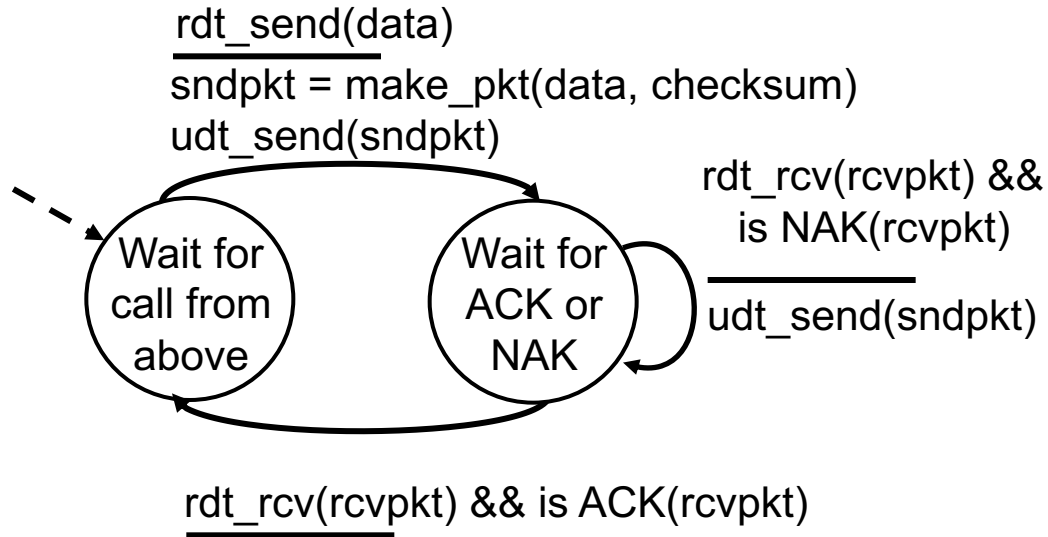
- underlying channel may flip bits in packet
 - checksum to detect bit errors
- *the question*: how to recover from errors:

*How do humans recover from “errors”
during conversation?*

rdt2.0: channel with bit errors

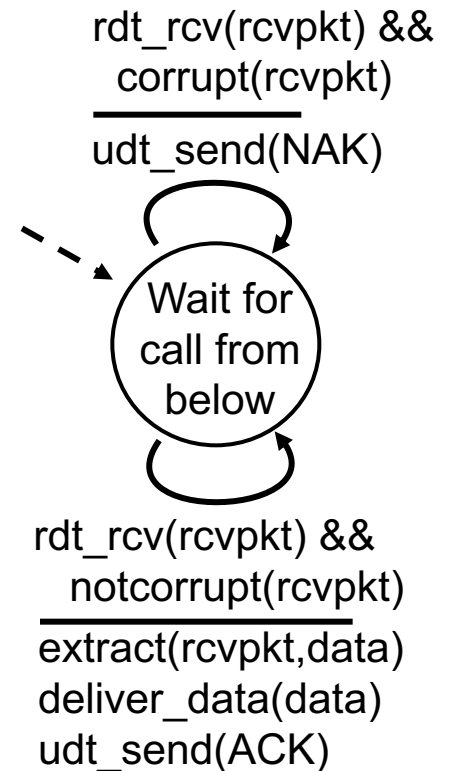
- underlying channel may flip bits in packet
 - checksum to detect bit errors
- *the question: how to recover from errors:*
 - *acknowledgements (ACKs)*: receiver explicitly tells sender that pkt received OK
 - *negative acknowledgements (NAKs)*: receiver explicitly tells sender that pkt had errors
 - sender retransmits pkt on receipt of NAK
- new mechanisms in `rdt2.0` (beyond `rdt1.0`):
 - error detection
 - feedback: control msgs (ACK,NAK) from receiver to sender

rdt2.0: FSM specification

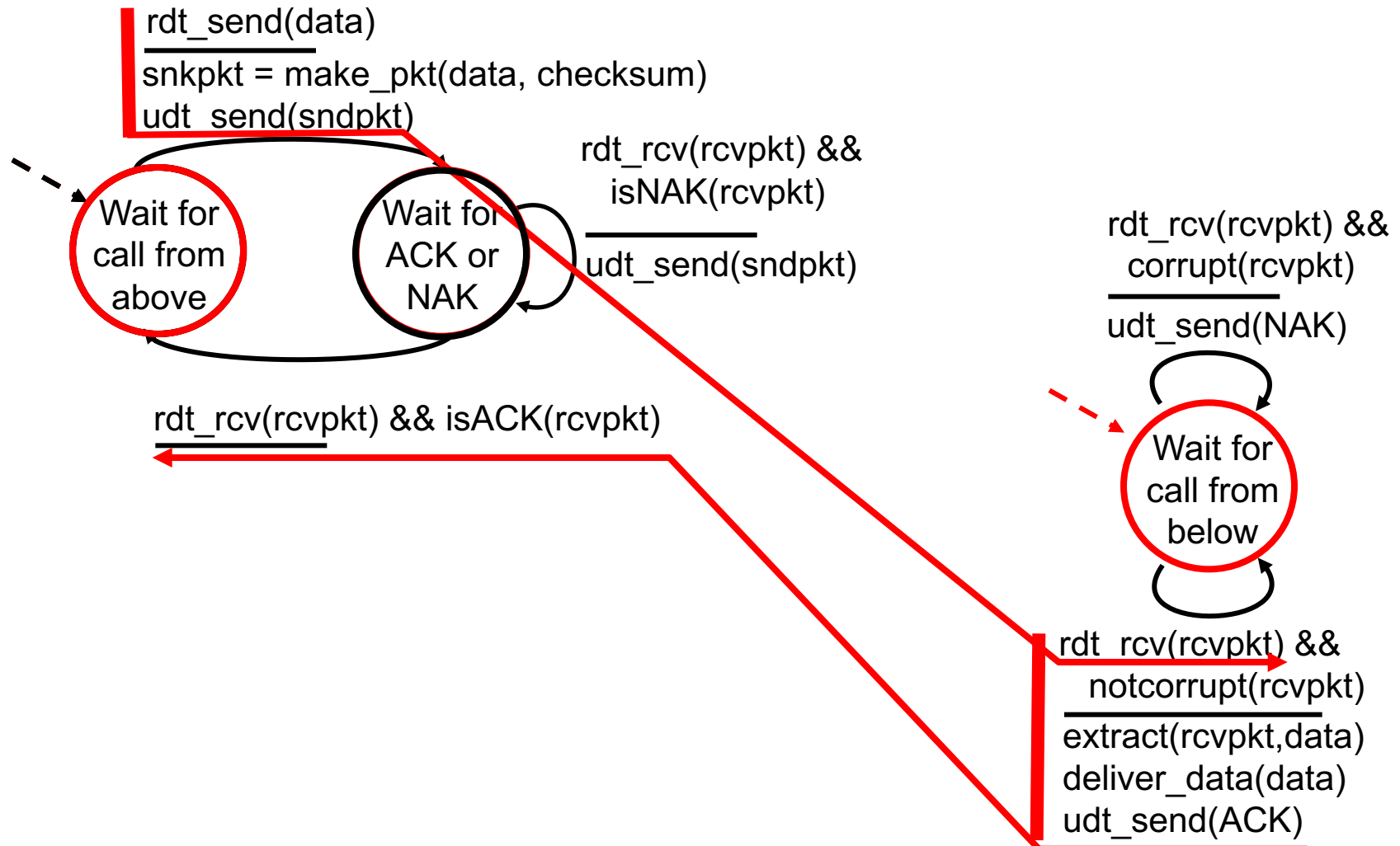


sender

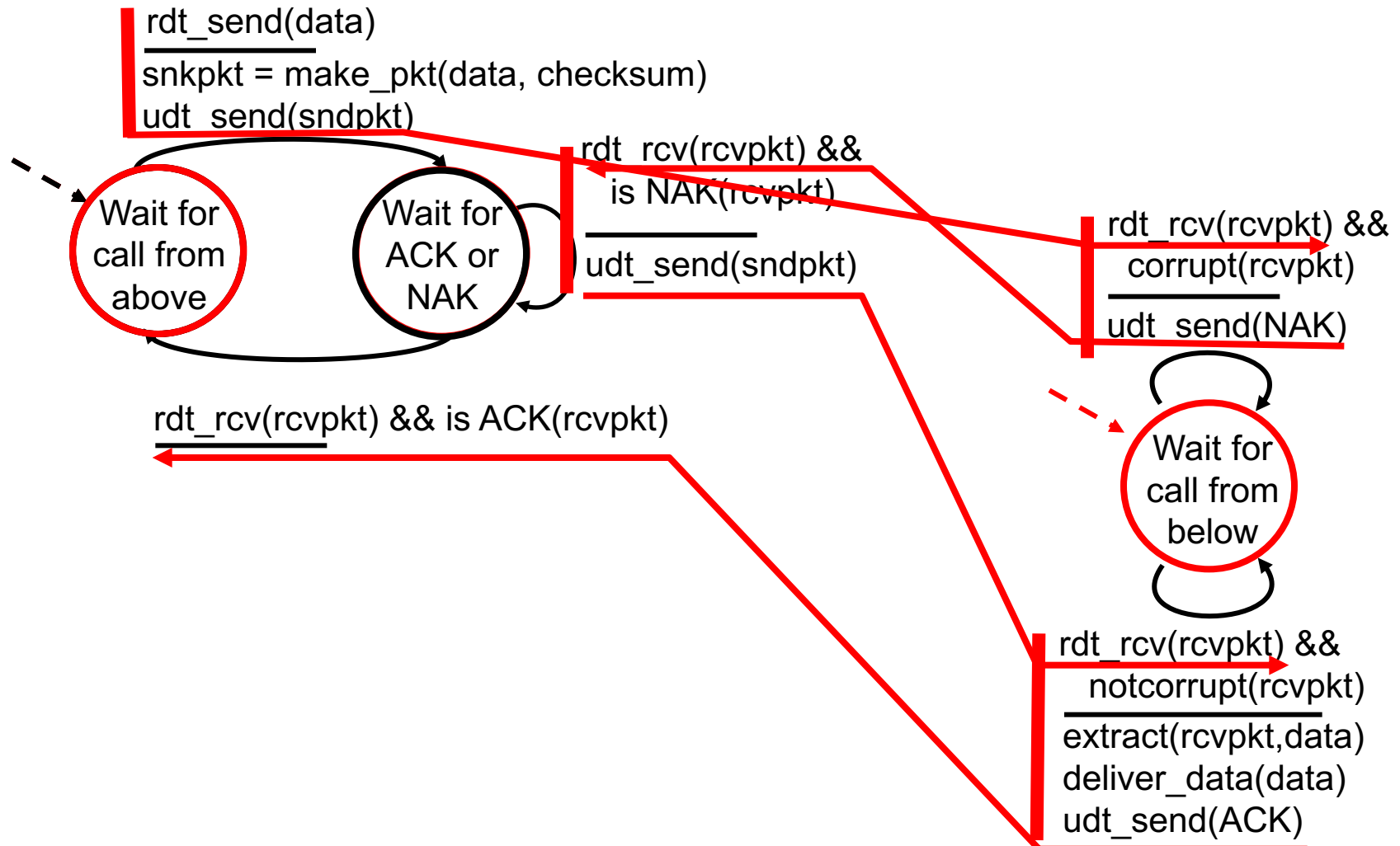
receiver



rdt2.0: operation with no errors



rdt2.0: error scenario



rdt2.0 has a fatal flaw!

what happens if ACK/NAK corrupted?

- sender doesn't know what happened at receiver!
- can't just retransmit: possible duplicate

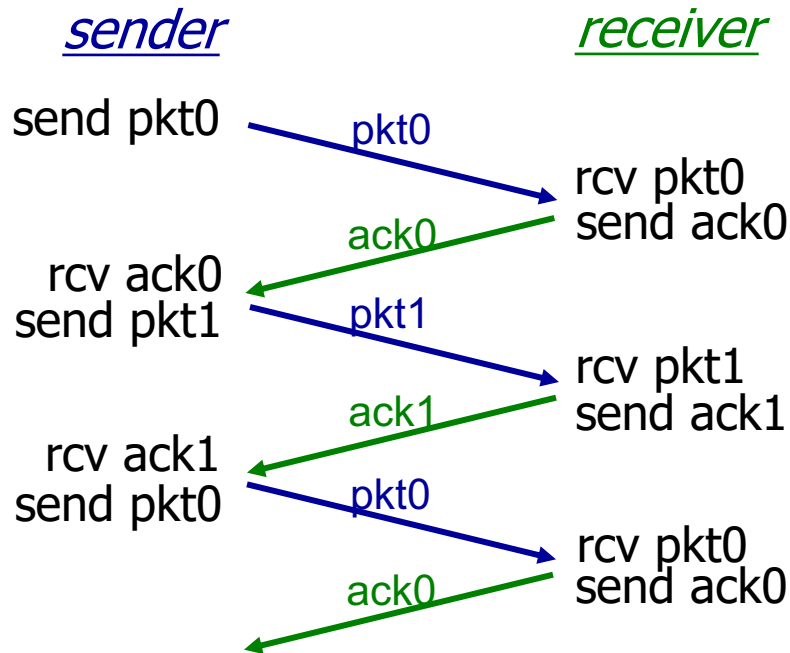
how to handling duplicates:

- sender retransmits current pkt if ACK/NAK corrupted
- sender adds *sequence number* to each pkt
- receiver discards (doesn't deliver up) duplicate pkt

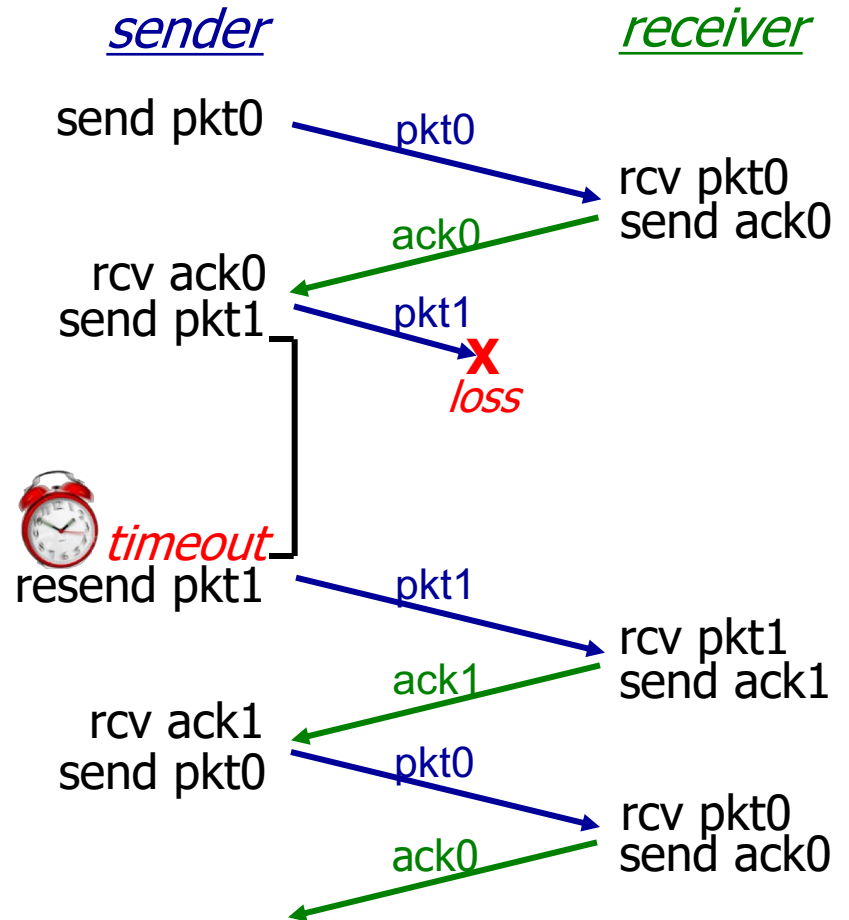
— stop and wait —

sender sends one packet,
then waits for receiver
response

Stop and Wait with Seq. Nos

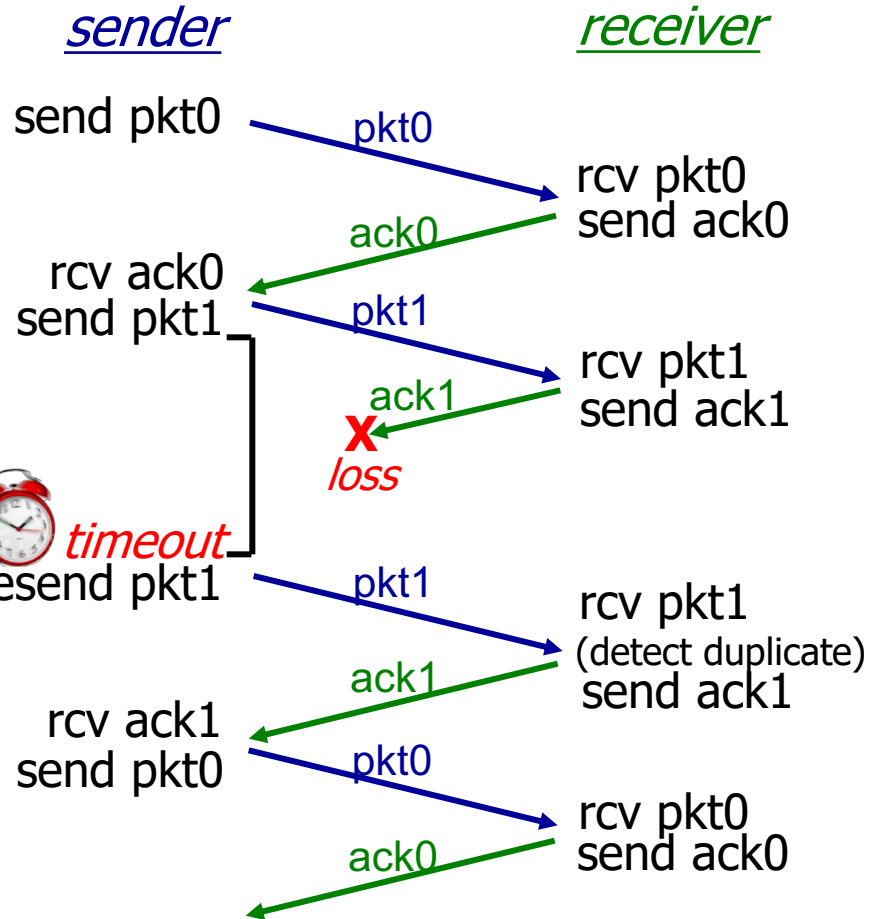


(a) no loss

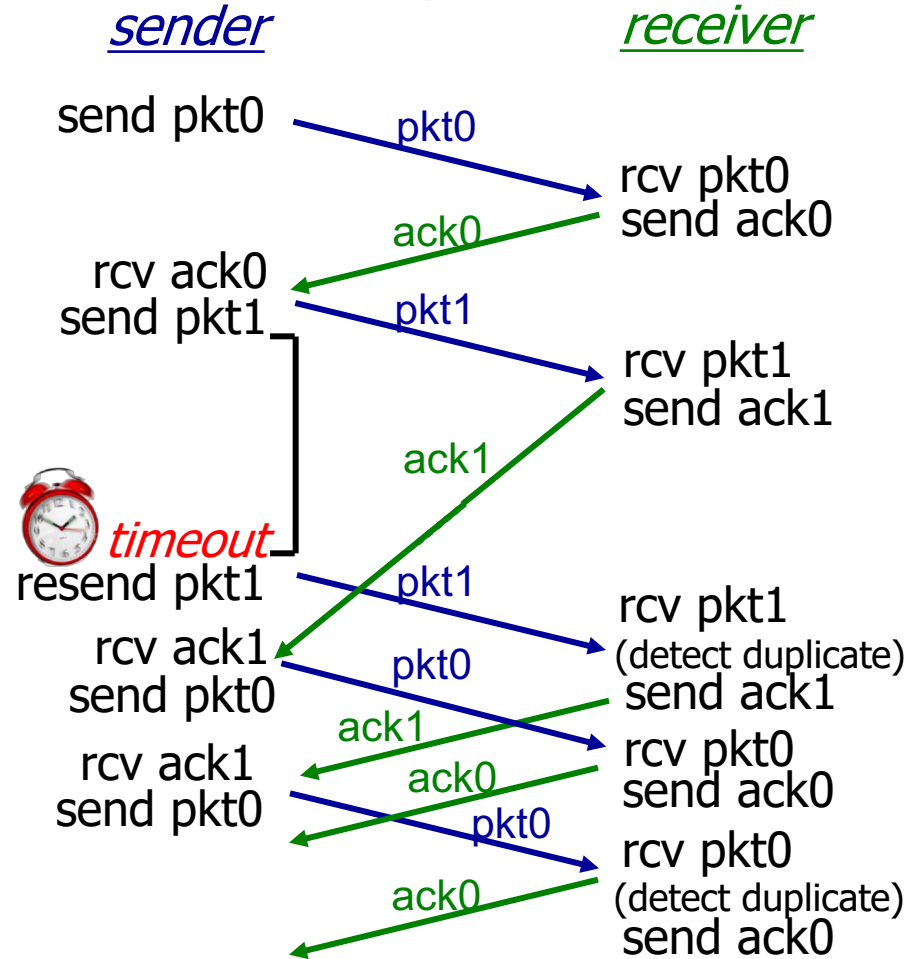


(b) packet loss

Stop and Wait with Seq. Nos



(c) ACK loss



(d) premature timeout/ delayed ACK

Performance of Protocol

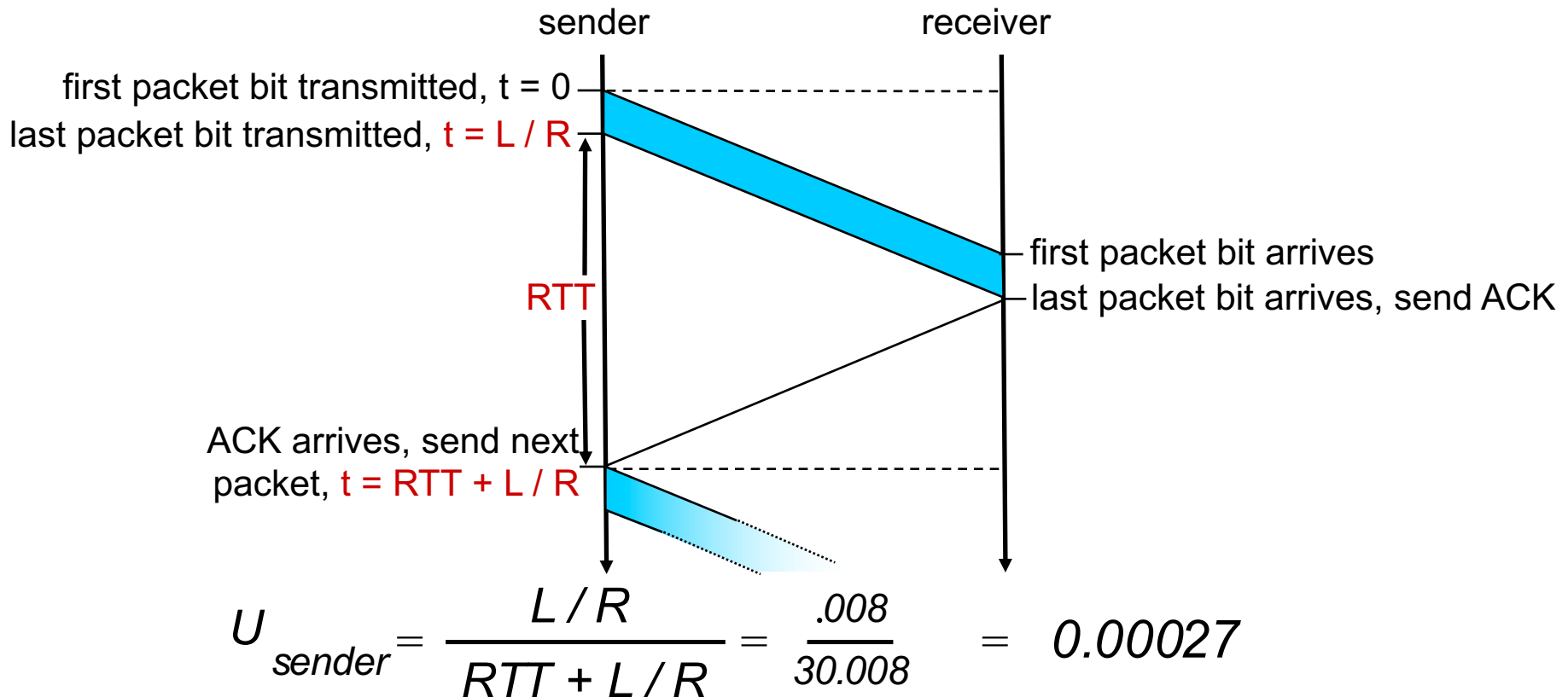
- it is correct, but performance is poor
- e.g.: 1 Gbps link, 15 ms prop. delay, 8000 bit packet:

$$D_{trans} = \frac{L}{R} = \frac{8000 \text{ bits}}{10^9 \text{ bits/sec}} = 8 \text{ microsecs}$$

- U_{sender} : *utilization* is fraction of time sender busy sending

$$U_{\text{sender}} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

rdt3.0: stop-and-wait operation

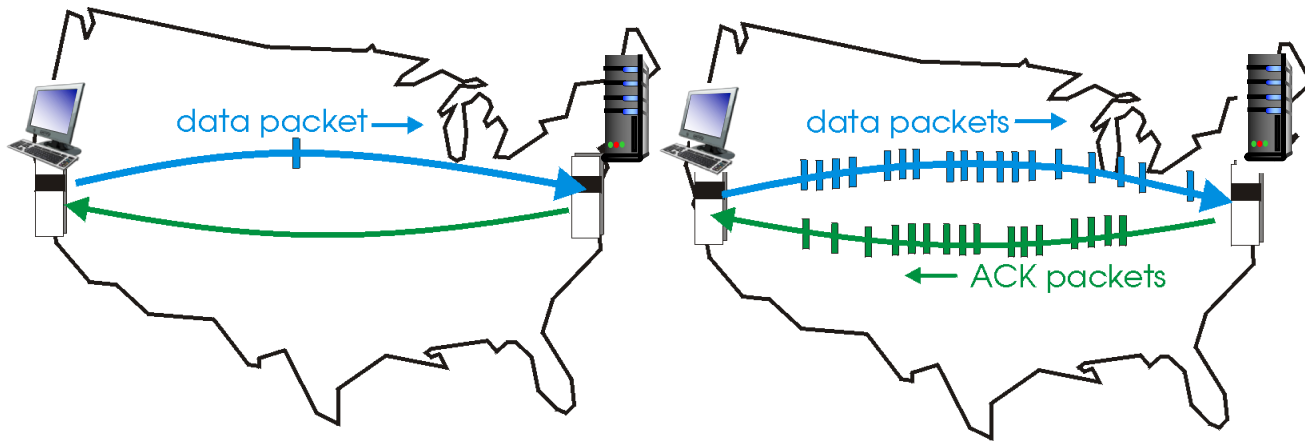


- if $RTT = 30$ msec, one 8KB pkt every 30 msec
→ ~265kB/sec throughput over 1 Gbps link
- network protocol limits use of physical resources!

Pipelined protocols

pipelining: sender allows multiple, “in-flight”, yet-to-be-acknowledged pkts (window of packets in transit)

- range of sequence numbers must be increased
- buffering at sender and/or receiver

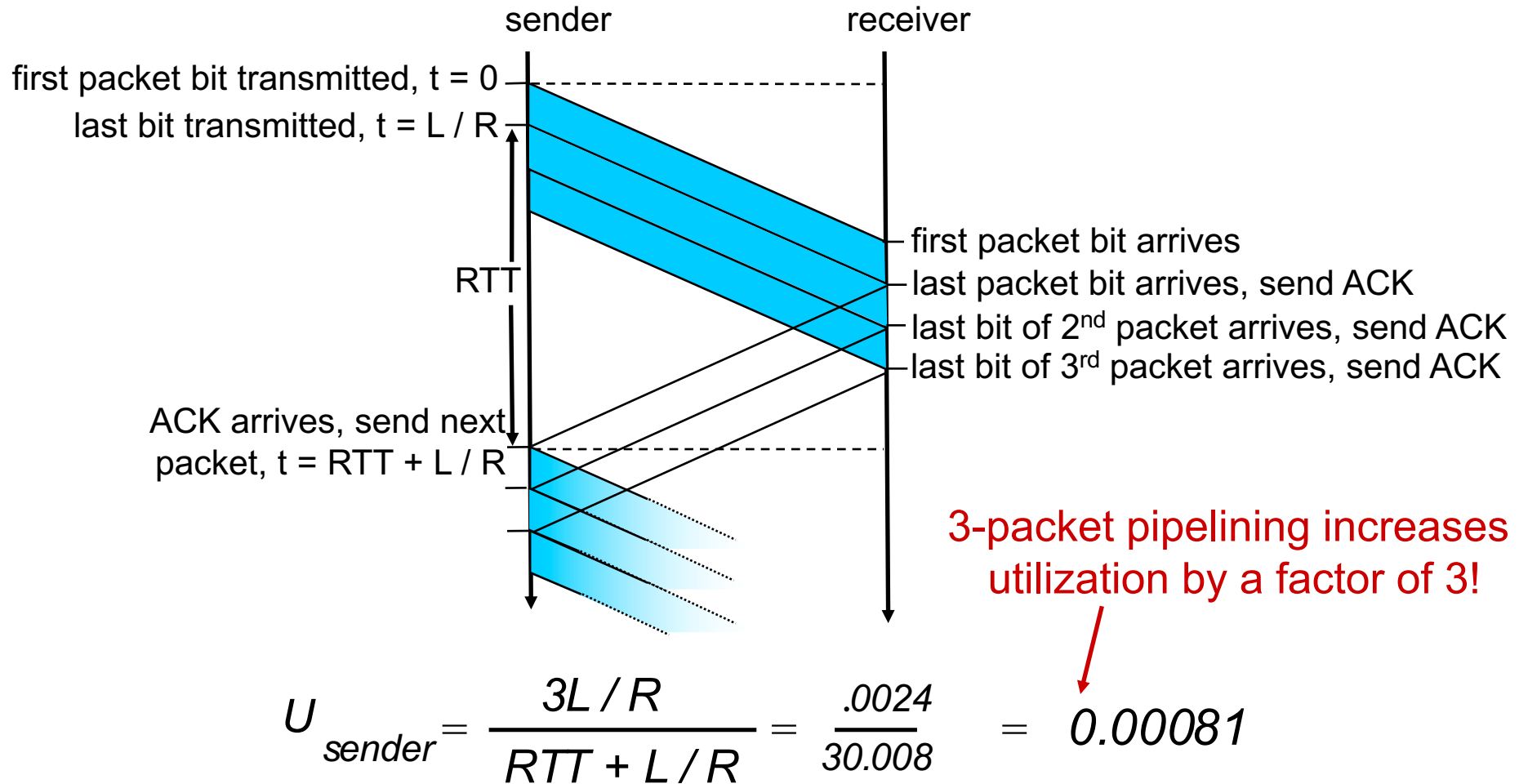


(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

- two generic forms of pipelined protocols: *go-Back-N (GBN)*, *selective repeat (SR)*

Pipelining: increased utilization



Pipelined protocols: overview

Go-back-N:

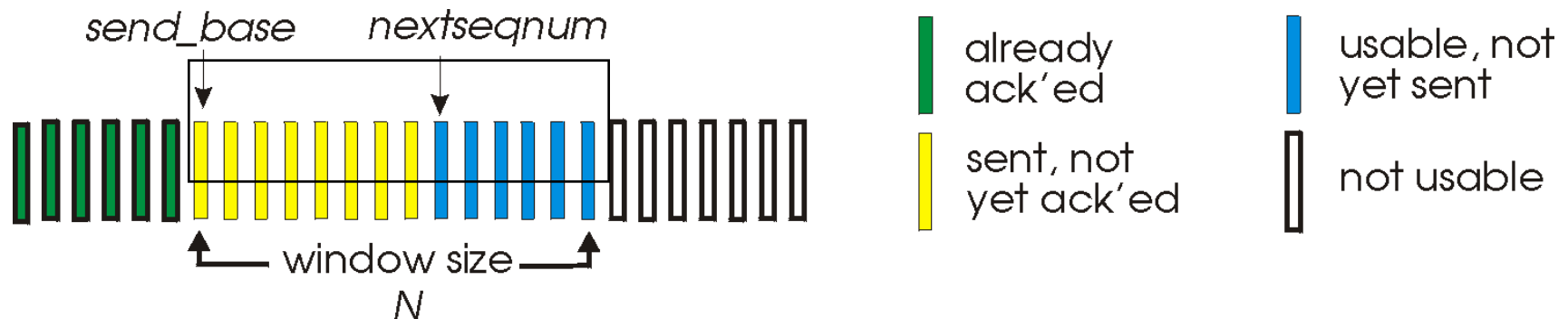
- sender can have up to N unacked packets in pipeline
- receiver only sends *cumulative ack*
 - doesn't ack packet if there's a gap
- sender has timer for oldest unacked packet
 - when timer expires, retransmit *all* unacked packets

Selective Repeat:

- sender can have up to N unack'ed packets in pipeline
- rcvr sends *individual ack* for each packet
- sender maintains timer for each unacked packet
 - when timer expires, retransmit only that unacked packet

Go-Back-N: sender

- k-bit seq # in pkt header
- “window” of up to N consecutive unack’ed pkts allowed between sender and receiver



- ACK(n): ACKs all pkts up to and including seq # n - *“cumulative ACK”* → *waiting for next packet with seq # n+1*
 - may receive duplicate ACKs (see receiver)
- timer for **oldest** in-flight pkt
- *timeout(n)*: retransmit packet n and **all** older seq # pkts in window (**note** window is all **“unack’ed”** packets)

GBN in action

sender window (N=4)

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

(wait – window full)

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

-GBN - only one timer
-tracks oldest pkt
-updated every ACK

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

Timer **reset**
for pkt(2)

pkt 2 timeout

GBN - resend all pkts
from (n=1 + 1) onwards

sender

send pkt0

send pkt1

send pkt2

send pkt3

(wait – window full)

rcv ack0, send pkt4

rcv ack1, send pkt5

ignore duplicate ACK

send pkt2

send pkt3

send pkt4

send pkt5

receiver

receive pkt0, send ack0

receive pkt1, send ack1

receive pkt3, discard,
(re)send ack1

receive pkt4, discard,
(re)send ack1

receive pkt5, discard,
(re)send ack1

GBN - discard all pkts from
(n=1) onwards, out of order

rcv pkt2, deliver, send ack2

rcv pkt3, deliver, send ack3

rcv pkt4, deliver, send ack4

rcv pkt5, deliver, send ack5

Timer **reset** after every received ACK for oldest unACKed pkt. Here last ACK was for pkt(1), so timer reset for unACKed pkt(2)

Selective repeat

- receiver *individually* acknowledges all correctly received pkts
 - buffers pkts, as needed, for eventual in-order delivery to upper layer
- sender only resends pkts for which ACK not received
 - sender timer for each unACKed pkt (actually oldest unACKed pkt, timer reset everytime a pkt is ACKed for oldest unACKed packet, all ACKed packets marked as received, and no timer reset if packets are not oldest)
- sender window
 - N consecutive seq #'s
 - Limit on seq #'s of sent, unACKed pkts (max window size is set for transmission)

Selective repeat: sender, receiver windows

